

VCU 模糊测试系统 — 完整项目计划 V2

更新日期：2026-05-21

V2.2 更新： VCU 仿真器扩展至 5 个功能模块（24条需求）；Test Plan 按 IEEE 829 标准重构；引入 ISO 9126 质量特性分类；用 Chapter 4 风险分析框架（Tech×Business Risk）为每个测试套件选择并论证技术。

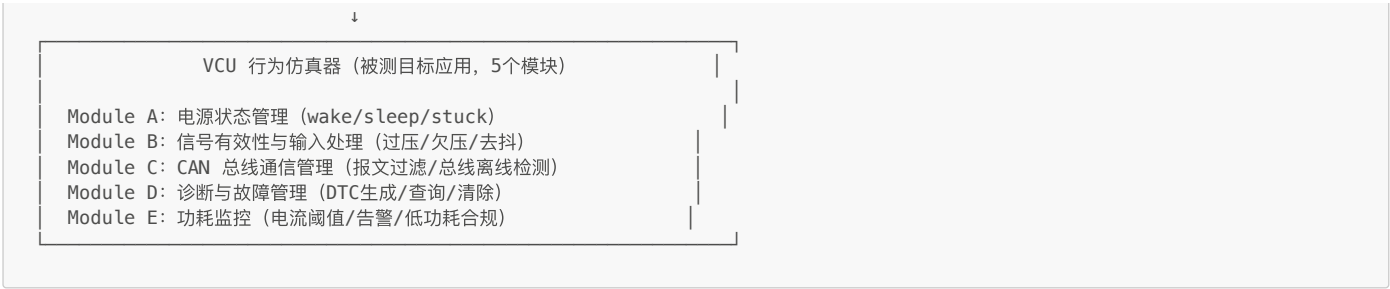
目录

- 一、项目定位与整体架构
 - 1.1 两个系统，职责明确
 - 1.2 为什么这样设计
- 二、VCU 行为仿真器设计
 - 2.1 总体状态机（Module A 核心）
 - 2.2 Module A — 电源状态管理
 - 2.3 Module B — 信号有效性与输入处理
 - 2.4 Module C — CAN 总线通信管理
 - 2.5 Module D — 诊断与故障管理
 - 2.6 Module E — 功耗监控
 - 2.7 VCU 仿真器完整 API
 - 2.8 完整需求列表（24条）
- 三、AutoTestDesign Tool 设计
 - 3.1 功能需求覆盖
 - 3.2 FR 1.0/1.1 — LLM 需求解析实现
 - 3.3 FR 2.0 — LLM 动态风险评估
 - 3.4 FR 3.0 — 测试用例生成
 - 3.5 Interactive Review 流程与 API
 - 3.6 非功能需求（NFR）规范
 - 3.7 工具整体 API 一览
 - 3.8 测试用例 JSON 格式（与 bq_new 兼容）
 - 3.9 两轮模糊测试反馈机制
- 四、测试计划（IEEE 829 Software Test Plan）
 - STP-1 引言
 - STP-2 测试项
 - STP-3 待测特性（ISO 9126 分类）
 - STP-4 质量风险分析（Tech × Business Risk RPN）
 - STP-5 测试方法（含技术选择论证）
 - 5.1 组件测试
 - 5.2 集成测试（含 Driver & Stub 识别）
 - 5.3 系统测试
 - 5.4 性能测试
 - 5.5 恢复测试
 - 5.6 回归测试
 - 5.7 白盒测试
 - 5.8 高层测试套件设计（10 Suite，139条用例 + Coverage Items 分类）
 - STP-6 通过/失败准则（含缺陷管理决策树）
 - STP-7 测试过程（Schedule / Tollgate / 分工 / 资源）
 - STP-8 环境需求
 - STP-9 变更管理
 - STP-10 计划批准
- 五、风险分析报告
 - 5.1 质量风险汇总表（RPN 评分）
 - 5.2 已知缺陷深入分析（QR-1.3，RPN=1）
- 六、开发工作量与时间安排
- 七、关键资料来源

一、项目定位与整体架构

1.1 两个系统，职责明确





1.2 为什么这样设计

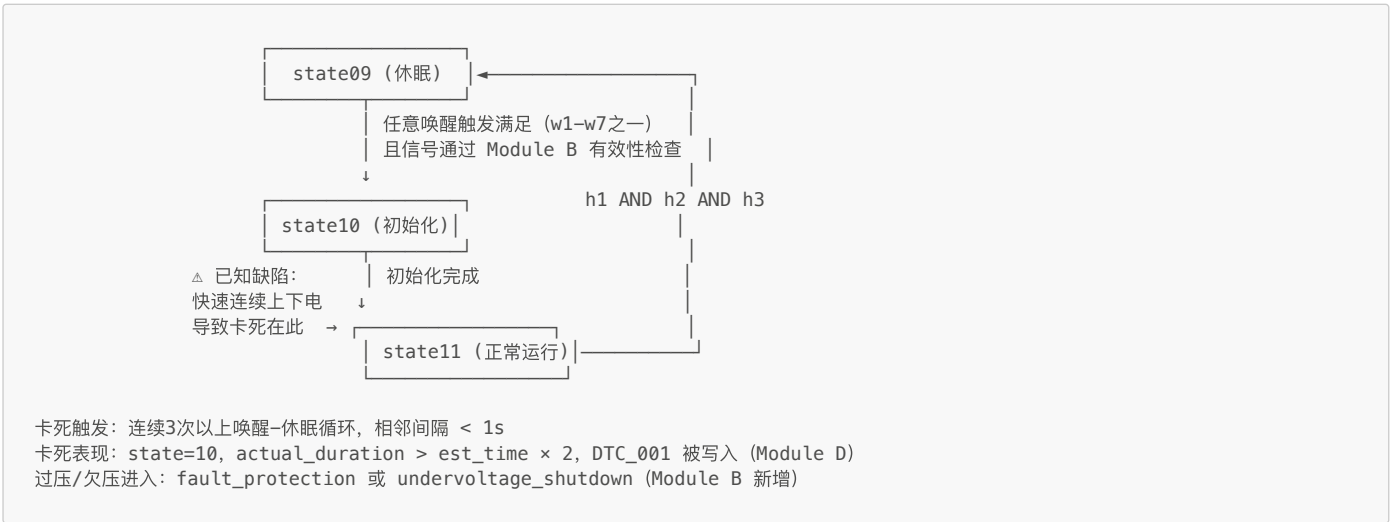
1. 5个模块均有真实工程依据：Module A 来自北汽需求文档；Module B 是任何汽车 ECU 必备的信号完整性保护；Module C 扩展自己有的 REQ-002/010；Module D 是 state10 卡死缺陷的自然延伸（实车 DTC）；Module E 基于仿真器已有的 power_current 输出字段。

2. 多模块 = 更多技术覆盖：5个模块覆盖 EP/BVA/决策表/状态转移/场景测试全部黑盒技术 + All-States/All-Transitions/Branch Coverage 白盒技术，充分体现课程知识。

3. ISO 9126 完整覆盖：Functionality（Module A/B/C）、Reliability（Module B/C/D）、Efficiency（Module A时序/Module E功耗），正好对应 ISO 9126 功能性测试与非功能性测试两个维度。

二、VCU 行为仿真器设计

2.1 总体状态机（Module A 核心）



2.2 Module A — 电源状态管理

唤醒触发条件（Type 1，单信号，任一满足即唤醒）

编号	信号名	触发条件	时序要求
w1	供电电压	voltage > 9.0V	duration ≥ 10ms
w2	CAN 网络报文	can_msg_id ∈ [0x400, 0x47F]	无
w3	CP 信号	cp_voltage > 9.0V	无
w4	CC 信号	cc_voltage < 4.4V	无
w5	CC2 信号	cc2_voltage < ubr_threshold	无
w6	口盖信号	hood_voltage > 4.0V	duration ≥ 10ms
w7	门板信号	door_voltage < 1.0V	duration ≥ 10ms

休眠触发条件（Type 2，多信号，必须同时满足）

编号	信号名	触发条件
h1	VCUIIdle_flg	VCUO_bDIAG_VCUIdle_flg = 1
h2	AuthComplete_flg	VCUO_bDIAG_AuthComplete_flg = 1
h3	CAN 停发	CAN 0x400-0x47F 停止发送

时序双条件判断（w1/w6/w7）

值条件	时序条件	判断结果	result_type
满足	duration ≥ 阈值	唤醒成功	expected
满足	duration < 阈值	时序不足	error (时序无效类)
不满足	任意	值不足	error (值无效类)

2.3 Module B — 信号有效性输入处理（新增）

Module B 在信号进入 Module A 状态机判断之前，先执行三项保护性检查。这是任何车用 ECU 必须实现的输入卫士（Input Guard）层。

过压保护（REQ-015）

- 触发：供电电压 > 16.0V（超过车载 12V 系统额定上限 + 33%余量）
- 响应：VCU 拒绝执行任何唤醒，进入 `fault_protection` 状态，`power_alarm_flag=1`
- 恢复：需通过 POST /reset 手动复位

欠压保护（REQ-016）

- 触发：供电电压 < 6.0V（低于 12V 系统最低工作电压）
- 响应：VCU 进入 `undervoltage_shutdown`，立即关闭 CAN 通信，`bus_message_flag=0`

信号去抖（REQ-017）

- 适用：w1/w6/w7（有时序要求的三路信号）
- 规则：duration < 5ms 的信号视为噪声，直接忽略，不触发任何状态转移
- 目的：防止线路瞬态干扰导致误唤醒

```
# vcu_simulator/modules/signal_guard.py

OVERVOLTAGE_THRESHOLD = 16.0 # V
UNDervOLTAGE_THRESHOLD = 6.0 # V
DEBOUNCE_MIN_MS = 5 # ms

def validate_signal(signal_name: str, value: float, duration_ms: int | None) -> dict:
    """
    Module B: 信号有效性检查，在进入 Module A 状态机前执行。
    返回 {"valid": bool, "reason": str, "fault_type": str | None}
    """
    # 过压检查（仅供电电压）
    if signal_name == "供电电压" and value > OVERVOLTAGE_THRESHOLD:
        return {"valid": False, "fault_type": "overvoltage",
                "reason": f"供电电压 {value}V 超过过压阈值 {OVERVOLTAGE_THRESHOLD}V"}
    # 欠压检查（仅供电电压）
    if signal_name == "供电电压" and value < UNDervOLTAGE_THRESHOLD:
        return {"valid": False, "fault_type": "undervoltage",
                "reason": f"供电电压 {value}V 低于欠压阈值 {UNDervOLTAGE_THRESHOLD}V"}
    # 去抖检查（w1/w6/w7）
    TIMING_SIGNALS = {"供电电压", "口盖信号", "门板信号"}
    if signal_name in TIMING_SIGNALS and duration_ms is not None and duration_ms < DEBOUNCE_MIN_MS:
        return {"valid": False, "fault_type": "debounce_rejected",
                "reason": f"持续时间 {duration_ms}ms < 去抖阈值 {DEBOUNCE_MIN_MS}ms, 判定为噪声"}
    return {"valid": True, "fault_type": None, "reason": "信号有效"}
```

2.4 Module C — CAN 总线通信管理（新增）

Module C 管理 CAN 通信层，REQ-002 和 REQ-010 都依赖 CAN 总线，需要独立的通信管理模块。

CAN 报文 ID 过滤（REQ-018）

- 规则：仅处理 ID ∈ [0x400, 0x47F] 的报文
- 超出范围的 ID：静默丢弃，不触发唤醒，不记录 DTC
- 报文边界：0x3FF 和 0x480 是测试用边界值

CAN 总线离线检测（REQ-019）

- 机制：维护错误计数器（error_counter），每次发送/接收失败 +1
- 阈值：error_counter > 255 → 进入 bus_off 状态，bus_message_flag 强制置 0
- 恢复：POST /reset 将 error_counter 清零，退出 bus_off

```
# vcu_simulator/modules/can_manager.py

CAN_VALID_ID_MIN = 0x400
CAN_VALID_ID_MAX = 0x47F
```

```
BUS_OFF_THRESHOLD = 255

class CANManager:
    def __init__(self):
        self.error_counter = 0
        self.bus_off = False

    def process_message(self, msg_id: int) -> dict:
        """Module C: CAN 报文处理, 返回是否触发唤醒"""
        if self.bus_off:
            return {"wake_triggered": False, "reason": "总线处于 bus_off 状态"}
        if not (CAN_VALID_ID_MIN <= msg_id <= CAN_VALID_ID_MAX):
            return {"wake_triggered": False, "reason": f"ID 0x{msg_id:03X} 超出有效范围"}
        return {"wake_triggered": True, "reason": f"ID 0x{msg_id:03X} 在有效范围内, 触发唤醒"}

    def report_error(self):
        self.error_counter += 1
        if self.error_counter > BUS_OFF_THRESHOLD:
            self.bus_off = True

    def reset(self):
        self.error_counter = 0
        self.bus_off = False
```

2.5 Module D — 诊断与故障管理（新增）

Module D 实现基本的 DTC（Diagnostic Trouble Code）管理，当 Module A 检测到 state10 卡死时，Module D 负责记录、存储和清除故障码。这是实车 ECU 的标准功能（参考 ISO 14229 UDS 诊断协议）。

DTC 故障码定义

DTC 代码	触发条件	说明
DTC_001	state10 stuck（REQ-012 触发）	快速上下电导致初始化卡死
DTC_002	过压保护触发（REQ-015）	供电电压超过 16V
DTC_003	欠压保护触发（REQ-016）	供电电压低于 6V

DTC 生命周期（状态转移）



DTC 相关 API

GET /dtc	查询所有存储的 DTC（含时间戳和触发次数）
POST /reset	复位 VCU 状态；加参数 {"clear_dtc": true} 时同时清除 DTC

```
# vcu_simulator/modules/dtc_manager.py
from datetime import datetime

class DTCManager:
    def __init__(self):
        self._dtcs: dict[str, dict] = {} # {dtc_code: {count, timestamps, status}}

    def log_dtc(self, code: str, reason: str):
        """REQ-020: 记录 DTC, 允许重复触发（计数累加）"""
        if code not in self._dtcs:
            self._dtcs[code] = {"count": 0, "first_seen": datetime.now().isoformat(),
                                "last_seen": None, "status": "active", "reason": reason}
        self._dtcs[code]["count"] += 1
        self._dtcs[code]["last_seen"] = datetime.now().isoformat()
        self._dtcs[code]["status"] = "active"

    def get_all(self) -> list[dict]:
        """REQ-021: 返回所有 DTC 记录"""
        return [{"code": k, **v} for k, v in self._dtcs.items()]

    def clear_all(self):
        """REQ-022: 清除所有 DTC"""
        for code in self._dtcs:
            self._dtcs[code]["status"] = "cleared"
```

2.6 Module E — 功耗监控（新增）

Module E 监控 power_current 字段（已在现有输出中），独立成模块以支持非功能测试。

电流阈值定义

状态	预期电流	告警阈值	ISO 9126 特性
state11（正常运行）	≤ 0.05A（50mA）	> 0.2A 持续 500ms → power_alarm_flag=1	Efficiency / Resource utilization
state09（休眠）	≤ 0.01A（10mA）	> 0.05A 视为休眠失败	Efficiency / Resource utilization
state10（卡死）	> 0.2A（实测约 250mA）	同时触发 DTC_001	Reliability / Maturity

```
# vcu_simulator/modules/power_monitor.py
import time

ALARM_THRESHOLD_A = 0.2 # A
ALARM_DURATION_MS = 500 # ms

class PowerMonitor:
    def __init__(self):
        self._high_current_start: float | None = None
        self.power_alarm_flag = 0

    def update(self, current_a: float):
        """REQ-023: 持续超过告警阈值 500ms 则置 power_alarm_flag=1"""
        now = time.time() * 1000 # ms
        if current_a > ALARM_THRESHOLD_A:
            if self._high_current_start is None:
                self._high_current_start = now
            elif now - self._high_current_start >= ALARM_DURATION_MS:
                self.power_alarm_flag = 1
        else:
            self._high_current_start = None
            self.power_alarm_flag = 0

    def check_sleep_compliance(self, current_a: float) -> bool:
        """REQ-024: 休眠状态功耗合规检查"""
        return current_a <= 0.01
```

2.7 VCU 仿真器完整 API

POST /simulate	执行一条测试用例（type1 或 type2）
POST /reset	复位 VCU 状态到 state09; {"clear_dtc": true} 同时清除 DTC
GET /state	查询当前 VCU 状态（含 power_alarm_flag）
GET /config	查询所有信号阈值（含 Module B/E 阈值）
PUT /config	修改信号阈值
GET /dtc	查询所有存储的 DTC (Module D)
GET /performance	查询性能监控报告 (actual_duration 统计)

POST /simulate 扩展响应体（新增 Module B/D/E 字段）：

```
{
  "vehicle_state": 11,
  "vehicle_mode": 2,
  "power_current": 0.042,
  "bus_message_flag": 1,
  "pdcu_wake_reason": 1,
  "actual_duration": 14.7,
  "result_type": "expected",
  "power_alarm_flag": 0,
  "bus_off_flag": 0,
  "active_dtcs": [],
  "signal_guard_result": {"valid": true, "fault_type": null},
  "detail": "供电电压=9.3V > 9V, 持续15ms ≥ 10ms, Module B 校验通过, 硬线唤醒成功"
}
```

2.8 完整需求列表（24条）

REQ-ID	模块	标题	描述	风险等级
--------	----	----	----	------

REQ-ID	模块	标题	描述	风险等级
REQ-001	A	硬线供电唤醒 w1	供电电压 > 9V 持续 ≥ 10ms 时唤醒，pdcu_wake_reason=1	High
REQ-002	A/C	CAN 网络唤醒 w2	CAN 总线收到 0x400~0x47F 报文时唤醒，pdcu_wake_reason=2	High
REQ-003	A	CP 信号唤醒 w3	CP 幅值 > 9V 上升沿触发唤醒，pdcu_wake_reason=3	Medium
REQ-004	A	CC 信号唤醒 w4	CC 电压下降至 < 4.4V 触发唤醒，pdcu_wake_reason=4	Medium
REQ-005	A	CC2 信号唤醒 w5	CC2 UBR 电压下降沿触发唤醒，pdcu_wake_reason=5	Medium
REQ-006	A	口盖信号唤醒 w6	口盖电压 > 4V 持续 ≥ 10ms 触发唤醒，pdcu_wake_reason=6	Low
REQ-007	A	门板信号唤醒 w7	门板电压 < 1V 持续 ≥ 10ms 触发唤醒，pdcu_wake_reason=7	Low
REQ-008	A	休眠条件 h1	VCUIdle_flg=1 是休眠必要条件之一	High
REQ-009	A	休眠条件 h2	AuthComplete_flg=1 是休眠必要条件之一	High
REQ-010	A/C	休眠条件 h3	CAN 0x400~0x47F 停发是休眠必要条件之一	High
REQ-011	A	三条件同时才休眠	h1 AND h2 AND h3 全满足时进入 state09，任一不满足维持当前状态	High
REQ-012	A/D	卡死缺陷检测	连续3次以上快速唤醒-休眠（间隔<1s）导致 state10 卡死，actual_duration 超时	High
REQ-013	A	输出字段一致性	state11 时 bus_message_flag=1；state09 时 bus_message_flag=0	Medium
REQ-014	A	响应时序合规	type1 actual_duration ≤ 20s；type2 actual_duration ≤ 60s	Medium
REQ-015	B	过压保护	供电电压 > 16V → fault_protection，拒绝唤醒，power_alarm_flag=1	High
REQ-016	B	欠压保护	供电电压 < 6V → undervoltage_shutdown，bus_message_flag=0	High
REQ-017	B	信号去抖	w1/w6/w7 信号 duration < 5ms 视为噪声，不触发状态转移	Medium
REQ-018	C	CAN ID 过滤	仅处理 0x400~0x47F 范围报文；超出范围静默丢弃	Medium
REQ-019	C	CAN 总线离线检测	error_counter > 255 → bus_off_flag=1，bus_message_flag=0	Medium
REQ-020	D	DTC 故障码生成	state10 卡死触发时写入 DTC_001；过压触发写入 DTC_002；欠压写入 DTC_003	High
REQ-021	D	DTC 信息查询	GET /dtc 返回所有 DTC，含代码/状态/触发次数/时间戳	Low
REQ-022	D	DTC 清除	POST /reset?clear_dtc=true 清除所有 DTC，状态置 cleared	Low
REQ-023	E	功耗告警	state11 下 power_current > 0.2A 持续 > 500ms → power_alarm_flag=1	Medium
REQ-024	E	低功耗合规	state09 下 power_current 必须 ≤ 0.01A	Medium

三、AutoTestDesign Tool 设计

核心定位：工具是通用的汽车 VCU 测试工具，而非专为北汽 VCU 定制。VCU 的 24 条需求是演示 Demo，工具应能处理任意粘贴进来的 VCU 类需求文本。

3.1 功能需求覆盖

FR	功能	实现说明	是否必须
FR 1.0	Input/Parsing	三种输入：① UI 文本框粘贴原始需求文本（LLM 解析）；② 上传 CSV；③ 加载 VCU Demo	必须
FR 1.1	Requirement Structuring	LLM 从原始文本提取 Input Fields/Conditions/Expected Actions；结果在 UI 中可编辑	必须
FR 2.0	Risk Analysis & Prioritization	LLM 动态计算 ISO 9126 质量特性 × 技术/业务风险，输出 RPN 和建议测试深度	必须
FR 3.0	Black-Box Test Design	LLM 生成 EP/BVA/决策表/状态转移/场景测试；每条附 Coverage Item ID 和 REQ-ID	必须
FR 4.0	White-Box Test Modeling	state09/10/11 状态机建模，All-States + All-Transitions 覆盖 ★加分	可选
FR 5.0	Test Oracle Generation	LLM 合成 5 个输出字段的预期值 ★加分	可选
FR 6.0	Output & Export	导出 JSON（bq_new 格式）/ CSV / Excel，含需求-用例追溯矩阵	必须
FR 7.0	Test Suite Optimization	基于 RPN 排序 + bq_new DataVariation 变异策略 ★加分	可选

3.2 FR 1.0/1.1 — LLM 需求解析实现

输入方式设计

用户操作（三选一）：

① 粘贴文本

→ React <textarea>

→ POST /api/requirements/parse

② 上传 CSV

→ 文件选择器

→ POST /api/requirements/upload-csv

③ 加载 Demo

→ 点击按钮

→ GET /api/requirements/demo

```
# tool/services/requirement_parser.py
import anthropic, json, time

client = anthropic.Anthropic()

PARSE_SYSTEM_PROMPT = """你是软件测试工程师，从原始需求文本中识别所有需求条目并结构化：
- id: 需求编号（自动生成 REQ-001 等）
- title, description
- input_fields: [{name, data_type, valid_range, unit, has_timing}]
- conditions: [{type(timing/logical/combined), description, threshold}]
- expected_actions: [{output_field, expected_value, operator(eq/gte/lte)}]
仅返回 JSON 数组，不添加任何额外文字。"""

def parse_requirements_from_text(raw_text: str) -> tuple[list[dict], float]:
    """单次批量 LLM 调用，控制总耗时，满足 NFR ≤2s 目标"""
    t = time.time()
    resp = client.messages.create(
        model="claude-sonnet-4-6", max_tokens=4096,
        system=PARSE_SYSTEM_PROMPT,
        messages=[{"role": "user", "content": raw_text}]
    )
    elapsed = time.time() - t
    content = resp.content[0].text
    return json.loads(content[content.find('['):content.rfind('')+1]), elapsed
```

3.3 FR 2.0 — LLM 动态风险评分（基于 ISO 9126 + Chapter 4 框架）

风险评分采用课件 Chapter 4 的 **Tech Risk × Business Risk** 框架，LLM 动态评估。

```
# tool/services/risk_analyzer.py
RISK_SYSTEM_PROMPT = """你是汽车ECU测试专家，按 ISO 9126 质量特性对需求进行风险评估。
对每条需求评估：
- iso9126_characteristic: Functionality/Reliability/Efficiency/Maintainability
- tech_risk: 1(Very High)~5(Very Low)，技术实现出错的可能性
- business_risk: 1(Very High)~5(Very Low)，失效对用户/安全的影响
- rpn: tech_risk × business_risk (1-25，越小越需要充分测试)
- extent: rpn 1-5→Extensive, 6-10→Broad, 11-15→Cursory, 16-25→Low priority
- reasoning: 1句说明

返回 JSON 数组，字段同上。"""
```

3.4 FR 3.0 — 测试用例生成

```
# tool/services/testcase_generator.py
TECHNIQUE_INSTRUCTIONS = {
    "EP": "等价类划分 (EP)：识别有效类/无效类/边界特殊类，每类至少1条",
    "BVA": "边界值分析 (BVA)：每个数值边界生成 below/on/above 三点",
    "DT": "决策表 (DT)：枚举所有条件组合 (≤8种穷举，否则最小覆盖集)",
    "ST": "状态转移 (ST)：All-Transitions 覆盖，每条迁移路径生成序列",
    "SC": "场景测试 (Scenario)：基于真实使用场景设计端到端流程用例",
}
```

3.5 Interactive Review 流程与 API（强制要求）

步骤① Concept	→ POST /api/requirements/parse → [用户编辑解析结果]
步骤② Coverage	→ GET /api/coverage-items → [用户增/删/改覆盖项]
步骤③ Strategy	→ PUT /api/strategies/{req_id} → [用户更改技术] → POST regenerate
步骤④ Test Cases	→ [用户编辑/删除/添加用例]
步骤⑤ Prompts	→ [用户修改 LLM Prompt → 重新生成]
步骤⑥ Results	→ POST /api/execute → GET /api/results/summary
步骤⑦ Improve	→ POST /api/improve → [确认新增 Coverage Item]

每步对应完整 CRUD API（详见 3.7 工具 API 一览）。

3.6 非功能需求（NFR）规范

NFR 1: 性能（强制，≤2秒）

场景	预期耗时	是否满足 NFR
单条需求 EP/BVA 生成	~1.0~1.8s	满足

场景	预期耗时	是否满足 NFR
单条需求全技术生成	~1.5~2.5s	临界，需实测
24条需求并发生成	~4~6s	超出，见文档分析
CSV 导入	<0.1s	满足

改进建议（文档中必须写明）：① 本地 LLM（Ollama）将延迟降至 <200ms；② 预生成 VCU Demo 缓存；③ 流式输出让用户感知立即响应。

NFR 2：安全性

```
# 文件上传校验
ALLOWED_EXTENSIONS = {".csv", ".txt", ".json"}
MAX_FILE_SIZE = 5 * 1024 * 1024 # 5MB
# API Key 通过环境变量注入，不硬编码
# LLM 响应只做 JSON 解析，不 eval/exec
```

NFR 3：可用性

React + Ant Design：7步向导；LLM 输出字段全部可内联编辑；加载动画 + 预计时间显示。

NFR 4：可维护性

层	技术	理由
前端	React + TypeScript + Ant Design	类型安全 + 现成组件
后端	Python FastAPI + Pydantic	异步 + 自动校验 + OpenAPI 文档
LLM	Anthropic SDK (claude-sonnet-4-6)	稳定 JSON 输出
DB	SQLite + SQLAlchemy	本地零部署

3.7 工具整体 API 一览

方法	路径	功能
POST	/api/requirements/parse	LLM 解析文本
POST	/api/requirements/upload-csv	CSV 导入
GET	/api/requirements/demo	VCU Demo
PUT	/api/requirements/{id}	编辑需求字段
POST	/api/requirements/analyze-risk	LLM 风险分析（含 RPN）
PUT	/api/requirements/{id}/risk	手动覆写风险
GET/POST/PUT/DELETE	/api/coverage-items/{id}	覆盖项管理
PUT	/api/strategies/{req_id}	更改测试技术
POST	/api/strategies/{req_id}/regenerate	重新生成用例
POST	/api/test-cases/generate	单条需求生成
POST	/api/test-cases/generate-all	并发批量生成
PUT/DELETE/POST	/api/test-cases/{id}	用例编辑
GET/PUT	/api/prompts/{type}	Prompt 管理
POST	/api/execute	执行测试套件
GET	/api/results/summary	覆盖率统计
POST	/api/improve	生成变异改进用例
GET	/api/export/{json csv excel}	导出
GET	/api/performance	NFR 性能报告

3.8 测试用例 JSON 格式（与 bq_new 兼容）

```
{
  "type": 1,
  "in_data": [
    { "name": "供电电压", "data_type": "float", "value": "9.3", "duration": 15 }
  ],
```



```
"expected_results": [
  {"name": "整车State状态", "out_type": 1, "out_range": 2, "value": 11},
  {"name": "总线报文发送标志位", "out_type": 1, "out_range": 2, "value": 1},
  {"name": "PDCU唤醒原因", "out_type": 1, "out_range": 2, "value": 1},
  {"name": "整车模式", "out_type": 1, "out_range": 2, "value": 2},
  {"name": "功耗电流", "out_type": 1, "out_range": 3, "value": 0.05}
],
"error": [
  {"error_type": 1, "out_data": [{"name": "整车State状态", "out_type": 1, "out_range": 2, "value": 10}]},
  {"error_type": 2, "out_data": [
    {"name": "整车State状态", "out_type": 1, "out_range": 2, "value": 10},
    {"name": "actual_duration", "out_type": 1, "out_range": 1, "value": 40}
  ]}
],
"est_time": 20
}
```

out_range=1:≥ / out_range=2:= / out_range=3:≤

3.9 两轮模糊测试反馈机制（来自 bq_new process_control.py）

第一轮：加载初始用例 → DataVariation 生成变异池 → 执行 → 发现 new_state/stuck → 入库
第二轮：从库中取第一轮新状态 → 针对性变异 → 再次执行 → 确认复现 / 找到更多边界

第一轮发现	第二轮动作	Assignment 映射
new_state	单/多参数变异	新增 Coverage Item
stuck（REQ-012）	重复执行 × N 次	缺陷证据
actual_duration 趋势升	边界搜索	非功能深入分析

四、测试计划（IEEE 829 Software Test Plan）

本节按 IEEE Std 829-1998 《Software Test Documentation》标准骨架编写。
风险分析采用 Chapter 4 课件的 **Tech Risk × Business Risk → RPN** 框架。
质量特性分类采用 **ISO 9126**（Chapter 1/2 课件）。
测试技术选择依据来自 **Chapter 4** 课件的技术适用准则。

STP-1 引言（Introduction）

STP-1.1 目标（Objectives）

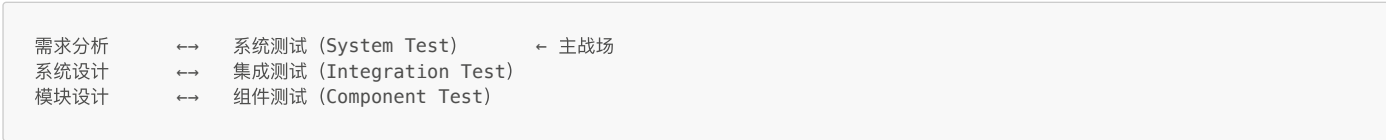
本软件测试计划（STP）的目标是：使用 AutoTestDesign Tool 对 **VCU 行为仿真器 v1.0** 实施系统化测试，验证其 5 个功能模块（Module A–E，共 24 条需求）满足 ISO 9126 质量特性要求，重点确保 High 风险需求（REQ-001/002/011/012/015/016/020）在发布前得到充分验证。

测试范围覆盖：

- **功能性（Functionality）**：Module A（状态管理）、Module B（信号有效性）、Module C（CAN 通信）的正确行为
- **可靠性（Reliability）**：已知 state10 卡死缺陷（REQ-012）的检测与 DTC 记录（Module D）
- **效率（Efficiency）**：响应时序（REQ-014）和功耗合规（Module E）

STP-1.2 测试策略（Testing Strategy）

采用 **V 模型**（Chapter 2 课件 2.1.1）组织测试层次：



核心策略：

- **风险驱动**：以 Chapter 4 的 RPN（Tech Risk × Business Risk）决定测试深度，RPN ≤ 5 进行 Extensive 测试
- **AutoTestDesign Tool 驱动**：工具自动生成黑盒用例并执行，人工设计白盒用例补充
- **两轮模糊测试**：第一轮执行初始用例集，第二轮基于发现的新状态/卡死进行针对性变异

STP-1.3 范围（Scope）

纳入测试：VCU 仿真器 5 个模块全部 HTTP API，共 24 条功能/非功能需求。

排除测试（Features NOT to be Tested，含理由）：

排除项	排除理由
硬件层信号采集	仿真器为纯软件，无物理 ADC 接口
OBD-II 物理总线	教学项目，不涉及物理 CAN 总线
多 VCU 网络拓扑	超出本项目范围，需要额外网络仿真基础设施
安全启动/固件加密	本仿真器不实现 HSM/加密引导
生产环境负载测试	仅进行功能正确性测试，不做大流量压力测试

STP-1.4 参考资料 (Reference Material)

文档	用途
北汽需求文档 0319.pdf	Module A 需求来源，已知 state10 卡死缺陷
北汽方案 0324.pdf	模糊测试框架、Δt 反馈机制
IEEE Std 829-1998	本测试计划骨架依据
ISO 9126:2001	质量特性分类依据
ISO/IEC/IEEE 29119-2	测试设计流程 (TD1~TD4) 依据
bq_new 学长代码	执行框架、结果判断逻辑
Assignment 2 PDF	交付物要求和评分标准

STP-1.5 术语与缩写 (Definitions and Acronyms)

术语	定义
VCU	Vehicle Control Unit，整车控制器
DTC	Diagnostic Trouble Code，诊断故障码
EP	Equivalence Partitioning，等价类划分
BVA	Boundary Value Analysis，边界值分析
DT	Decision Table，决策表
ST	State Transition，状态转移测试
SC	Scenario Testing，场景测试
RPN	Risk Priority Number = Tech Risk × Business Risk
SUT	System Under Test，被测系统 (VCU 仿真器)

STP-2 测试项 (Test Items)

STP-2.1 程序模块 (Program Modules)

模块 ID	模块名称	版本	需求范围	API 前缀
MOD-A	电源状态管理	v1.0	REQ-001~014	POST /simulate (type1/2)
MOD-B	信号有效性与输入处理	v1.0	REQ-015~017	POST /simulate (信号校验层)
MOD-C	CAN 总线通信管理	v1.0	REQ-018~019	POST /simulate (type1 w2)
MOD-D	诊断与故障管理	v1.0	REQ-020~022	GET/POST /dtc, POST /reset
MOD-E	功耗监控	v1.0	REQ-023~024	GET /state (power_alarm_flag)

STP-2.2 接口文档

VCU 仿真器通过 REST API 对外暴露所有功能，接口文档由 FastAPI 自动生成（访问 <http://localhost:8001/docs>），包含每个端点的输入模式、输出模式和错误码。

STP-3 待测特性 (Features to Be Tested)

按 ISO 9126 质量特性分类 (Chapter 1 课件)：

3.1 功能性 (Functionality) — 功能测试

ISO 9126 子特性	测试目标	覆盖模块	需求
Suitability (适合性)	VCU 是否正确处理 7 路唤醒触发	MOD-A	REQ-001~007

ISO 9126 子特性	测试目标	覆盖模块	需求
Suitability	h1/h2/h3 三条件 AND 逻辑是否正确	MOD-A	REQ-008~011
Accuracy（准确性）	电压/时序边界判断是否精确	MOD-A, MOD-B	REQ-001/006/007/015/016
Interoperability（互操作性）	CAN 报文 ID 过滤是否正确	MOD-C	REQ-018/019
Compliance（遵从性）	输出字段一致性满足规格	MOD-A	REQ-013

3.2 可靠性（Reliability）— 功能测试

ISO 9126 子特性	测试目标	覆盖模块	需求
Maturity（成熟性/健壮性）	VCU 对过压/欠压/去抖信号能否正确拒绝	MOD-B	REQ-015~017
Fault Tolerance（容错性）	CAN 总线离线时系统能否保持稳定	MOD-C	REQ-019
Recoverability（可恢复性）	卡死缺陷触发后 DTC 记录是否完整；手动复位后能否恢复	MOD-D	REQ-012/020~022

3.3 效率（Efficiency）— 非功能测试

ISO 9126 子特性	测试目标	覆盖模块	需求
Time Behaviour（时间特性）	actual_duration 是否满足时序规格	MOD-A	REQ-014
Resource Utilisation（资源利用）	各状态下 power_current 是否在规格范围内	MOD-E	REQ-023~024

3.4 可维护性（Maintainability）— 非功能测试

ISO 9126 子特性	测试目标	覆盖模块	需求
Analysability（可分析性）	DTC 代码是否准确标识故障类型和触发时间	MOD-D	REQ-020~021

STP-4 质量风险分析（Quality Risk Analysis）

遵循 Chapter 4 课件的风险分析框架：

- **Tech Risk（技术风险）**：实现出错的可能性（1=Very High, 5=Very Low）
- **Business Risk（业务风险）**：失效对用户/安全的影响（1=Very High, 5=Very Low）
- **RPN = Tech Risk × Business Risk**（1~25，越小越需要 Extensive 测试）
- **测试深度**：RPN 1~5=Extensive（广泛），6~10=Broad（全面），11~15=Cursory（一般），16~25=Low Priority

质量风险（Quality Risk）	ISO 9126 特性	Tech Risk	Bus Risk	RPN	测试深度	主要技术
1.0 电源状态管理						
1.1 唤醒触发条件判断错误	Functionality/Suitability	2	1	2	Extensive	EP + BVA
1.2 休眠多条件 AND 逻辑错误	Functionality/Suitability	2	1	2	Extensive	Decision Table
1.3 state10 卡死缺陷未检测	Reliability/Maturity	1	1	1	Extensive	Sequence + Fuzz
1.4 输出字段不一致	Functionality/Compliance	3	2	6	Broad	State Transition
1.5 时序 actual_duration 超限	Efficiency/Time	3	2	6	Broad	Performance
2.0 信号有效性						
2.1 过压/欠压保护失效	Reliability/Maturity	2	1	2	Extensive	EP + BVA
2.2 信号去抖逻辑错误	Reliability/Maturity	3	2	6	Broad	BVA（时序边界）
3.0 CAN 通信管理						
3.1 CAN ID 过滤范围错误	Functionality/Interoperability	3	2	6	Broad	EP + BVA
3.2 总线离线检测失效	Reliability/Fault Tolerance	3	2	6	Broad	Decision Table
4.0 诊断与故障管理						
4.1 DTC 未在卡死时生成	Reliability/Recoverability	3	2	6	Broad	Scenario Testing
4.2 DTC 清除后残留	Reliability/Recoverability	3	3	9	Broad	State Transition
5.0 功耗监控						
5.1 告警阈值判断错误	Efficiency/Resource	3	2	6	Broad	BVA
5.2 休眠功耗不合规	Efficiency/Resource	3	3	9	Broad	EP

STP-5 测试方法（Approach）

本节为每种测试类型规定方法、技术选择依据和工具。**技术选择依据均来自 Chapter 4 课件。**

STP-5.1 组件测试（Component Testing）

目的：验证每个模块（MOD-A~E）的内部逻辑单元正确实现。

模块	测试对象	技术	选技术依据（Chapter 4）
MOD-A	wake_trigger 判断函数	EP + BVA	输入为连续数值（电压 V，时间 ms），存在明确等价类划分和数值边界 → EP 识别类别，BVA 精确测试边界点
MOD-B	signal_guard.validate()	EP + BVA	三个保护阈值（6V/16V/5ms）均为数值边界 → BVA 生成 below/on/above 三点覆盖每个边界
MOD-C	can_manager.process_message()	EP + BVA	CAN ID 是离散整数区间 [0x400, 0x47F]，区间端点和外部值是高风险边界 → BVA 覆盖 0x3FF/0x400/0x47F/0x480
MOD-D	dtc_manager.log_dtc()	EP	DTC 状态为离散集合（not_set/active/cleared），每种状态是独立等价类
MOD-E	power_monitor.update()	BVA	0.2A/500ms 双重阈值均为数值边界，BVA 生成 below/on/above 覆盖两个维度的边界

框架：pytest 单元测试，mock 外部依赖。

STP-5.2 集成测试（Integration Testing）

目的：验证模块间接口和数据流正确，重点测试 Module B 的 Input Guard 对 Module A 状态机的拦截效果。

Driver 与 Stub 识别（Chapter 2 课件）：

在集成测试中，被测模块（MUT）的上下游需要用 Driver 和 Stub 代替尚未就绪的组件：

Driver（驱动器）= 主动调用 MUT 的测试组件
Stub（桩）= 被 MUT 调用的依赖组件的模拟替代

被测模块（MUT）	Driver（谁来调用它）	Stub（它依赖什么）	备注
VCU 仿真器（全体）	pytest 测试脚本（httpx）	无（仿真器是自包含系统）	仿真器不调用外部服务
AutoTestDesign Tool 后端	pytest + httpx 测试脚本	LLM API Stub （ <code>pytest-mock</code> mock <code>client.messages.create</code> ）； VCU 仿真器 Stub （httpx mock）	Tool 调用两个外部依赖，集成测试中均需打桩
Tool FR 1.0/1.1（LLM 解析）	单元测试脚本	LLM Stub ：预定义 JSON 响应，覆盖正常解析/格式错误/空响应三种场景	与 Omninet 课件 BankAuthorizationStub 同等模式

```
# LLM Stub 示例（与 Chap2DrivernStubOmninet 课件模式一致）
class LLMStub:
    """模拟 Anthropic API 响应，供 Tool 集成测试使用。"""
    SCENARIOS = {
        "vcu_normal": '[{"id":"REQ-001","title":"硬线唤醒","description":"..."}]',
        "malformed_json": "这不是JSON",
        "empty_response": "[]",
    }

    def create(self, model, max_tokens, system, messages):
        scenario = messages[0]["content"][:10] # 根据输入前缀路由
        mock_resp = MagicMock()
        mock_resp.content = [MagicMock(text=self.SCENARIOS.get("vcu_normal", "[]"))]
        return mock_resp
```

集成路径与测试场景：

集成路径	测试场景	技术
MOD-B → MOD-A	过压信号不触发 state09→state10 转移	State Transition（验证 invalid transition 不发生）
MOD-C → MOD-A	无效 CAN ID 不触发唤醒	EP（无效 ID 等价类）
MOD-A → MOD-D	state10 卡死时 DTC_001 正确写入	Scenario（完整卡死触发场景）
MOD-A → MOD-E	state09 下 power_current 满足 0.01A 上限	BVA（电流边界）
Tool LLM → Tool 解析层	LLM Stub 返回 malformed JSON，Tool 返回 400	EP（异常输入等价类）

框架：pytest + httpx 向真实 VCU 仿真器发 HTTP 请求；Tool 集成测试使用 pytest-mock 打桩 LLM。

STP-5.3 系统测试 (System Testing)

目的：使用 AutoTestDesign Tool 端到端验证所有 24 条需求，模拟真实测试工程师工作流程。

测试流程：

1. 将 24 条需求文本粘贴进 Tool → LLM 解析
2. 调整风险评分 (RPN 校验)
3. Tool 生成 10 个 Test Suite (见 STP-5.10 高层套件设计)
4. 执行全部套件，收集 result_type 统计
5. 触发第二轮模糊测试，验证 REQ-012 卡死可稳定复现

STP-5.4 性能测试 (Performance Testing)

ISO 9126 特性：Efficiency / Time Behaviour

测试目标：

- VCU 仿真器：actual_duration ≤ 20s (type1) / ≤ 60s (type2)
- AutoTestDesign Tool：单条需求用例生成时间记录 (目标 ≤ 2s, 超出时分析原因)

工具：pytest --benchmark 插件 + Tool 内置 /api/performance 端点。

STP-5.5 恢复测试 (Recovery Testing)

ISO 9126 特性：Reliability / Recoverability

测试 Module D 的故障恢复能力：

1. 触发 state10 卡死 → 验证 DTC_001 生成
2. POST /reset?clear_dtc=true → 验证 DTC 状态变为 cleared
3. 执行正常唤醒 → 验证系统恢复到正常工作状态 (state09→state11)

STP-5.6 回归测试 (Regression Testing)

每次 VCU 仿真器代码变更后，按 RPN 优先级选择需要重跑的 Suite：RPN ≤ 5 (Extensive) 的 Suite A/B/C/D/E/F 全量重跑；RPN 6~10 的 Suite G/H/I 仅重跑受影响的集成路径 (通过需求-用例追溯矩阵确定)。使用 pytest CI 集成自动触发。

STP-5.7 白盒测试 (Structure-Based Testing)

目的：通过检查 VCU 仿真器内部代码结构，补充黑盒测试无法覆盖的路径。

覆盖准则	目标	覆盖对象	对应 Suite
All-States	覆盖 state09 / state10 / state11 / fault_protection / undervoltage_shutdown	MOD-A + MOD-B 状态机	Suite D
All-Transitions	覆盖全部 7 条状态转移 (含卡死和两种故障保护)	同上	Suite D + F
Branch Coverage	7路唤醒 × 有效/无效/时序无效 = 21条分支	MOD-A wake_trigger	Suite A
Condition Coverage	Module B 三重条件 (过压/欠压/去抖) 各真/假分支	MOD-B validate_signal	Suite F

STP-5.8 高层测试套件设计 (High-Level Test Suite Design)

每个 Test Suite 遵循 IEEE 829 Test Design Specification 格式 (Chapter 4 课件 p.17)：包含 Suite ID / Features to be Tested / Approach & Technique / Tools / Pass/Fail Criteria / RPN 追溯。

Suite	覆盖需求	ISO 9126 特性	测试技术	技术选择理由 (Ch.4)	用例数	RPN 源
Suite A: 唤醒 EP	REQ-001~007	Functionality/Suitability	Equivalence Partitioning	7路信号各有有效/无效/时序无效三个等价类，EP 是划分输入域最系统的方法；避免冗余用例	21	RPN 2 (1.2)
Suite B: 唤醒 BVA	REQ-001/006/007/015/016	Functionality/Accuracy & Reliability/Maturity	Boundary Value Analysis	电压和时序阈值是连续数值，边界点处缺陷率最高 (off-by-one)；BVA 在每个边界生成 below/on/above 三点，是检测边界错误最高效的技术	24	RPN 2 (1.2)
Suite C: 休眠 决策表	REQ-008~011	Functionality/Suitability	Decision Table	h1 AND h2 AND h3 是 3 个布尔条件的组合，决策表能系统枚举所有 2³=8 种组合，防止遗漏某个角组合	8	RPN 2 (1.2)

Suite	覆盖需求	ISO 9126 特性	测试技术	技术选择理由（Ch.4）	用例数	RPN 源
Suite D：状态转移	REQ-001~012/015/016	Reliability/Maturity	State Transition Testing (All-Transitions)	VCU 核心是有限状态机（Finite-State Model, Ch.4 p.18）；All-Transitions 准则确保每条迁移至少执行一次，白盒和黑盒兼顾	15	RPN 1 (1.5)
Suite E：卡死专项	REQ-012/020	Reliability/Recoverability	Scenario Testing + Sequence	卡死缺陷是特定操作序列触发的真实 bug；Scenario Testing（Ch.4 p.3）专为“有意义的真实使用场景”设计，序列测试直接复现缺陷条件	10	RPN 1 (1.5)
Suite F：信号保护 EP+BVA	REQ-015~017	Reliability/Maturity	EP + BVA 组合	过压/欠压保护类似黑盒 EP（3个区间：正常/过压/欠压）+BVA（6V/16V 两个精确边界）；Ch.4 课件明确展示 EP 和 BVA 可组合使用（p.14, BVA+Decision Table 示例）	18	RPN 2 (2.5)
Suite G：CAN 通信	REQ-018~019	Functionality/Interoperability	EP + BVA + Decision Table	CAN ID 范围 [0x400, 0x47F] 是离散整数边界 → BVA；总线状态（normal/error/bus-off）× 报文合法性 → 决策表；三种技术组合体现 Ch.4 "mixing techniques" 思想	12	RPN 6 (3.3.2)
Suite H：诊断 DTC	REQ-020~022	Reliability/Recoverability & Maintainability/Analysability	Scenario Testing + State Transition	DTC 生命周期（not_set→active→cleared）是典型有限状态机 → State Transition；DTC 查询场景是真实诊断工程师 workflow → Scenario Testing	9	RPN 6 (4.4.2)
Suite I：功耗监控	REQ-023~024	Efficiency/Resource Utilisation	EP + BVA	power_current 分为三个等价类（正常/告警/危险）→ EP；0.01A/0.05A/0.2A 是三个数值边界 → BVA；恰好对应 Ch.1 课件"non-functional test addresses Efficiency"的典型场景	12	RPN 6+9 + 5.2
Suite J：性能时序	REQ-014	Efficiency/Time Behaviour	Performance Testing	响应时序是 ISO 9126 Efficiency 子特性，属于 Ch.1 课件"非功能测试覆盖 Efficiency"范畴；测量 actual_duration 并与规格阈值比对	10	RPN 6 (1.5)
合计	REQ-001~024	Functionality/Reliability/Efficiency/Maintainability	EP/BVA/DT/ST/SC/Performance	—	139	—

Coverage Items 分类示例（Suite A — w1 供电电压）

参照 Chapter 4 workshop（Printer Testing Coverage Items）的 Input/Output/Behavior/Environment 四类等价类格式，为 Suite A 建立完整 Coverage Item 追溯表：

Class ID	类型	参数	等价类描述	代表值	Test Case ID
I-W1-1	Input（有效类）	供电电压 + 时序	voltage > 9V AND duration ≥ 10ms	9.3V, 15ms	TC-A-001
I-W1-2	Input（无效类1）	供电电压	voltage ≤ 9V（电压不足）	8.9V, 15ms	TC-A-002
I-W1-3	Input（无效类2）	时序	voltage > 9V AND duration < 10ms（时序不足）	9.3V, 8ms	TC-A-003
I-W1-4	Input（特殊无效类）	供电电压 + 时序	voltage ≤ 9V AND duration < 10ms	8.5V, 5ms	TC-A-004
O-STATE-1	Output	vehicle_state	唤醒成功: state09 → state11	vehicle_state=11	TC-A-001
O-STATE-2	Output	vehicle_state	唤醒失败: state 维持 state09	vehicle_state=9	TC-A-002~004
O-REASON-1	Output	pdcu_wake_reason	w1 成功时 reason=1, 失败时 reason=0	1 / 0	TC-A-001~004
B-NORM-1	Behavior	正常唤醒序列	state09→state10→state11 完整三步序列	REQ-001 完整路径	TC-A-001
B-FAIL-1	Behavior	唤醒失败行为	收到无效信号后状态维持，无 DTC 生成	维持 state09	TC-A-002

Class ID	类型	参数	等价类描述	代表值	Test Case ID
E-SIM-1	Environment	VCU 仿真器版本	v1.0, POST /simulate, localhost:8001	当前版本	TC-A-ALL

7路唤醒（w1~w7）各有类似结构的 Coverage Items，合计 21 个 Input 等价类；每个 Class ID 追溯到具体测试用例，满足 IEEE 829 Test Design Specification 的可追溯性要求。

STP-6 通过/失败准则（Pass/Fail Criteria）

STP-6.1 暂停准则（Suspension Criteria）

采用 Omninet AI Master Test Plan 课件中的缺陷管理决策树逻辑（Chapter 5 workshop）：



触发暂停的具体条件（满足任一）：

- Critical/High（RPN ≤ 5）缺陷阻塞了 >28 条（>20%）剩余用例的执行
- VCU 仿真器 POST /simulate 连续返回 HTTP 500 超过 3 次（平台级故障）
- state09/10/11 状态机核心逻辑完全失效（Suite D All-Transitions 全部失败）

STP-6.2 恢复准则（Resumption Criteria）

暂停后，恢复流程（来自 Omninet 课件）：



具体验收：

- Blocker 缺陷修复并通过开发者单元测试
- VCU 仿真器重新部署，GET /state 返回 200，state=9（初始状态）
- Smoke Test：Suite D 的 T-001（正常唤醒）和 T-002（正常休眠）两条路径通过

STP-6.3 批准准则（Approval Criteria）

测试完成并批准发布的标准：

风险级别（RPN）	通过率要求
Extensive（RPN 1-5，对应 High 需求）	100% 通过
Broad（RPN 6-10，对应 Medium 需求）	≥ 95% 通过
Cursory 及以下（RPN 11+，对应 Low 需求）	≥ 80% 通过
性能 NFR（actual_duration）	type1 ≤ 20s, type2 ≤ 60s 全通过
功耗 NFR（power_current）	所有状态下满足 Module E 规格

STP-7 测试过程（Testing Process）

STP-7.1 测试交付物（Test Deliverables）

交付物	内容	负责人
本测试计划（STP）	IEEE 829 格式，本文档	测试负责人
风险分析报告	ISO 9126 分类 + RPN 评分	测试负责人
测试用例 JSON 文件	139 条用例（10 Suite），bq_new 格式	测试工程师
pytest 测试脚本	10 Suite 自动化执行脚本	测试工程师
测试结果报告	每条用例 Pass/Fail + Coverage Item 覆盖率	测试工程师
NFR 性能分析报告	actual_duration 统计 + Tool 生成时间分析	测试工程师

交付物	内容	负责人
缺陷记录	发现的所有缺陷（含已知 state10 卡死证据）	全组

STP-7.2 测试任务与时间安排（Schedule or Checklist）

周次	测试级别	主要任务	验收标准
W1	组件测试	MOD-A 单元测试（wake_trigger/sleep_cond）	所有 21 条 Branch Coverage 分支通过
W1	组件测试	MOD-B 单元测试（signal_guard）	6/16V 边界点，5ms 去抖边界全通过
W1	组件测试	MOD-C/D/E 单元测试	各模块独立函数 pytest 全绿
W2	集成测试	MOD-B→A 信号拦截路径	过压/欠压信号不触发 state 跳转
W2	集成测试	MOD-A→D DTC 写入路径	卡死后 GET /dtc 返回 DTC_001
W2	集成测试	MOD-A→E 功耗监控路径	state09 下 power_current ≤ 0.01A
W3	系统测试	Suite A（EP，21条）执行	expected/error 分类符合预期
W3	系统测试	Suite B（BVA，24条）执行	边界点判断准确
W3	系统测试	Suite C（决策表，8条）执行	C-008 为唯一 expected，其余 error
W3	系统测试	Suite D（状态转移，15条）执行	All-Transitions 7条路径全覆盖
W3	系统测试	Suite E（卡死，10条）执行	state10 stuck 稳定复现
W3	系统测试	Suite F（信号保护，18条）执行	过压/欠压/去抖全部正确拦截
W3	系统测试	Suite G（CAN，12条）执行	边界 ID 和总线离线正确处理
W3	系统测试	Suite H（DTC，9条）执行	DTC 生命周期完整覆盖
W3	系统测试	Suite I（功耗，12条）执行	告警阈值和休眠功耗符合规格
W3	非功能测试	Suite J（性能，10条）执行	actual_duration 全在规格内
W3	非功能测试	Tool NFR 性能测量	记录实测时间，超 2s 时写分析报告
W4	回归测试	第二轮模糊测试	卡死缺陷稳定复现，提交 DTC 证据
W4	验收	导出 JSON/CSV/Excel	格式与 bq_new 兼容，追溯矩阵完整
W4	验收	AutoTestDesign Tool 演示	7步 Interactive Review 流程可走通

STP-7.2.1 Tollgate 准则（Entry / Continuation / Exit Criteria）

参考 Omninet AI Master Test Plan 课件中的 Tollgate 机制：每个测试级别需通过入口标准（Entry）才能开始，满足连续标准（Continuation）才能继续，达到出口标准（Exit）才能进入下一级。

测试级别	Entry Criteria（进入条件）	Continuation Criteria（继续条件）	Exit Criteria（退出条件）
Tollgate 1：组件测试（W1）	pytest 测试框架就绪；各模块代码完成静态评审（STP-5.0）	本地开发环境稳定，无 import 错误	所有模块单元测试 ≥ 90% branch coverage ；High-RPN 模块（MOD-A/B）100% 通过
Tollgate 2：集成测试（W2）	Tollgate 1 通过；各模块组件测试全绿	每次合并代码不破坏已通过的集成路径	全部集成路径测试通过；HTTP 接口响应延迟 ≤ 100ms； 0 个未解决的接口缺陷
Tollgate 3：系统测试（W3）	Tollgate 2 通过；VCU 仿真器完整部署（5个模块均可调用）	缺陷修复交付时间 ≤ 48小时；无 >1小时的系统停机	100% Suite A~J 用例执行 ；RPN 1~5 用例 100% 通过；RPN 6~10 用例 ≥ 95% 通过
Tollgate 4：验收（W4）	Tollgate 3 通过；导出文件验证通过	Interactive Review 7步可完整走通	AutoTestDesign Tool 演示成功；JSON 导出与 bq_new 格式兼容；第二轮模糊测试缺陷证据完整

STP-7.3 职责（Responsibilities）

角色	人员	职责
Test Lead	（填写）	制定测试计划，风险分析，协调各子任务
VCU 开发	（填写）	实现 VCU 仿真器 5 个模块，修复缺陷
Tool 后端开发	（填写）	LLM 服务层 + Interactive Review API
Tool 前端开发	（填写）	React 7步向导界面
Test Engineer	（填写）	编写 139 条用例 JSON + pytest 脚本，执行测试
Documentation	（填写）	风险报告 + 测试结果报告 + NFR 分析

STP-7.4 资源（Resources）

资源类型	规格
开发机	macOS/Linux, Python 3.11+, Node.js 18+
VCU 仿真器端口	localhost:8001 (FastAPI)
Tool 后端端口	localhost:8000 (FastAPI)
Tool 前端端口	localhost:3000 (React dev server)
LLM API	Anthropic claude-sonnet-4-6 (ANTHROPIC_API_KEY 环境变量)
数据库	SQLite (本地文件, 无需安装)

STP-8 环境需求 (Environmental Requirements)

STP-8.1 硬件

标准开发笔记本/台式机 (x86-64 或 ARM Mac) , 无需特殊硬件。

STP-8.2 软件

软件	版本	用途
Python	3.11+	VCU 仿真器 + Tool 后端
FastAPI + uvicorn	最新版	HTTP 服务框架
pytest + httpx	最新版	自动化测试执行
React + TypeScript	18+	Tool 前端
Anthropic SDK	最新版	LLM 集成
SQLite	内置	测试数据持久化

STP-8.3 安全

- Anthropic API Key 通过环境变量注入, 不提交到 Git
- 文件上传限制扩展名 (.csv/.txt/.json) 和大小 (5MB)
- FastAPI CORS 限制为 localhost

STP-8.4 工具

工具	用途
AutoTestDesign Tool (本项目工具)	生成测试用例、执行并收集结果
pytest	执行测试脚本
httpx	HTTP 客户端 (测试脚本中调用 VCU API)
pytest-benchmark	性能测试
openpyxl	Excel 导出

STP-8.5 风险与假设 (Risks and Assumptions)

风险	影响	缓解措施
LLM API 延迟 > 2s	Tool NFR 不满足	预生成 VCU Demo 缓存; 文档分析原因
Anthropic API 不可用	无法执行 Tool 功能	使用 mock LLM 进行测试演示
VCU 仿真器开发延迟	无法执行系统测试	先完成 Tool 单元测试; 并行开发

STP-9 变更管理 (Change Management Procedures)

本 STP 版本控制:

- 每次修改需更新文档头部日期和版本号
- 需求变更须同步更新对应的 Test Suite 和用例 JSON
- 所有变更记录在 Git commit message 中, 格式: `test-plan: <变更摘要>`

STP-10 计划批准 (Plan Approvals)

角色	姓名	签名	日期
Test Lead	(填写)		2026-05-21

角色	姓名	签名	日期
项目负责人（填写）			

五、风险分析报告（Risk Analysis Report）

本报告针对被测目标应用（VCU 行为仿真器），不是对工具本身的分析。
采用 Chapter 4 课件的 ISO 9126 + Tech×Business Risk RPN 框架。

5.1 质量风险汇总表

QR-ID	质量风险描述	ISO 9126 特性	Tech Risk	Bus Risk	RPN	测试深度	追溯需求
QR-1.1	唤醒触发条件判断错误（7路信号之一）	Functionality	2	1	2	Extensive	REQ-001~007
QR-1.2	休眠三条件 AND 逻辑实现错误	Functionality	2	1	2	Extensive	REQ-008~011
QR-1.3	state10 卡死缺陷无法被检测	Reliability/Maturity	1	1	1	Extensive	REQ-012
QR-2.1	过压/欠压保护阈值判断失效	Reliability/Maturity	2	1	2	Extensive	REQ-015/016
QR-1.4	输出字段一致性被破坏	Functionality/Compliance	3	2	6	Broad	REQ-013
QR-1.5	actual_duration 超过时序规格	Efficiency/Time	3	2	6	Broad	REQ-014
QR-2.2	去抖逻辑错误导致噪声触发唤醒	Reliability/Maturity	3	2	6	Broad	REQ-017
QR-3.1	CAN ID 过滤边界实现错误	Functionality/Interoperability	3	2	6	Broad	REQ-018
QR-3.2	CAN 总线离线检测失效	Reliability/Fault Tolerance	3	2	6	Broad	REQ-019
QR-4.1	卡死时 DTC 未生成	Reliability/Recoverability	3	2	6	Broad	REQ-020
QR-5.1	功耗告警阈值判断错误	Efficiency/Resource	3	2	6	Broad	REQ-023
QR-4.2	DTC 清除操作不完整	Reliability/Recoverability	3	3	9	Broad	REQ-022
QR-5.2	休眠功耗超过 0.01A 规格	Efficiency/Resource	3	3	9	Broad	REQ-024
QR-4.3	DTC 查询接口返回格式错误	Maintainability/Analysability	4	3	12	Cursory	REQ-021

5.2 已知缺陷深入分析（QR-1.3, RPN=1, 最高优先级）

- 来源：北汽原始需求文档 0319.pdf，真实工程缺陷记录
- 缺陷描述：PDCU 在频繁上下电过程中偶发 state 卡死在 state10，控制器不休眠，功耗电流始终 > 200mA（REQ-012 + REQ-023 联合暴露）
- 触发条件：连续快速上下电（频率 > 1次/秒，持续 3 次以上）
- 根因：控制器底层状态机在高频切换时进入死锁分支，无法跳出 state10 初始化循环
- 测试策略：Suite E（Sequence + Fuzz）专项复现；Module D DTC_001 作为客观证据
- 缺陷证据：result_type = "stuck" + actual_duration > est_time × 2 + GET /dtc 返回 DTC_001

六、开发工作量与时间安排

模块	主要工作	估计天数
VCU 仿真器 Module A	state 机器 + 7路唤醒 + 3路休眠 + stuck检测 + FastAPI	3天
VCU 仿真器 Module B	signal_guard（过压/欠压/去抖）集成到 simulate 流程	1天
VCU 仿真器 Module C	CAN ID 过滤 + error_counter + bus_off 状态	1天
VCU 仿真器 Module D	dtc_manager + /dtc API + /reset 扩展	1天
VCU 仿真器 Module E	power_monitor + power_alarm_flag 集成到响应体	0.5天
Tool 后端 LLM 服务层	requirement_parser + risk_analyzer + testcase_generator	3天
Tool 后端 Interactive Review API	review.py 全接口 + db.py + executor.py	2天
Tool 前端 React	7步向导界面 + 可编辑表格 + 导出	3天
测试用例 JSON 文件	139条用例（10 Suite），bq_new 格式	2天
pytest 测试脚本	10 Suite 脚本 + LLM mock 单元测试	1.5天
NFR 性能测量与报告	实测 + 分析 + 改进建议文档	0.5天
文档写作	本计划 + 风险报告 + 测试结果报告	2天
合计		20.5天

七、关键资料来源

资料	用于
北汽需求文档 0319.pdf	Module A 7路唤醒条件、已知 state10 卡死缺陷
北汽方案 0324.pdf	模糊测试框架（M^N 输入空间）、Δt 反馈机制
北汽方案 0331.pptx	输入输出清单、测试平台接口对接方式
bq_new/services/data_init.py	测试用例 JSON 格式
bq_new/services/data_variation.py	三种变异策略（单参数/多参数/重复执行）
bq_new/services/result_judge.py	结果判断逻辑、5个输出字段
IEEE Std 829-1998	Test Plan 骨架（Section 四结构来源）
ISO 9126:2001	质量特性分类（Functionality/Reliability/Efficiency/Maintainability）
ISO/IEC/IEEE 29119-2	测试设计流程（TD1~TD4），Test Design Specification 格式
Chapter 4 课件（刘琴，同济大学）	Tech×Business Risk RPN 框架，EP/BVA/DT/ST 技术选择准则
Assignment 2 PDF	交付物要求、NFR 规范、评分标准
Anthropic SDK 文档	claude-sonnet-4-6 接入方式